四川大學

课程论文

论文题目：AI 数据分析的可执行校验

课程名称：新型软件技术

学　　院：

专业名称：

学生姓名：BAHODIR KODIRJON UGLI NAZAROV

学　　号：Omitted from public copy

任课教师：胡大裟博士

2026–2027 学年第 1 学期

摘　要

大语言模型生成的数据分析可能同时包含计算错误、单位错误、有效样本数错误和图表数据不一致。本文实现 TrustVerify，依据人工确认的分析契约，对结构化分析结果执行独立重算与逐字段校验。系统支持六类描述性统计操作，并在无法支持的输入上拒绝给出校验结论。实验区分受控错误注入与真实本地模型调用：前者用于检验校验层的覆盖范围，后者比较初始生成、自我检查及可执行反馈修正。参考答案由独立的 SQLite 实现产生，生产校验器使用 Decimal 运算。受控实验中，完整校验器检测到全部 288 个预设结构化错误，对 48 个正确报告未产生误报；但全部 48 个仅含无依据因果叙述的报告仍通过结构化检查。真实模型实验中，1.5B 模型接受反馈后，24 项任务中有 19 项的数值映射正确，但完整报告仍全部失败。该结果说明，通过数值与格式校验不等同于整篇报告可信。本文同时公开模型原始响应、实验设置及复现代码，讨论契约错误、叙述真实性、模型能力与小样本评估等限制。

关键词：大语言模型；数据分析；可执行校验；错误修正；可复现评估

Abstract

AI-generated analyses can contain arithmetic errors while presenting plausible prose. TrustVerify recomputes six supported descriptive operations from a human-approved contract and checks metric values, units, valid-row counts and chart data. We distinguish a controlled mutation study from actual local-model generation. Sixty synthetic datasets provide 12 development and 48 test tasks, with an independent SQLite oracle. Across 336 injected faulty reports, full verification detects 288 supported structured faults and misses all 48 unsupported causal narratives, with no false alarms on 48 clean reports. An actual-model pilot compares initial generation, one self-review call and one executable-feedback repair call on 24 paired tasks per model. The 0.5B model achieves no fully correct structured reports in any generation condition. The 1.5B model also produces no fully correct reports, although executable feedback corrects the numeric metric mapping on 19 of 24 tasks in an exploratory decomposition. Deterministic recomputation is sufficient for the approved contracts and is reported separately from model improvement. The project contributes a working prototype, field-level ablations, preserved raw calls and explicit reliability boundaries.

Keywords: language models; data analysis; executable verification; feedback; reproducibility

Contents

1 Introduction **4**

2 Related work **5**
2.1 Evaluating data-science systems 5
2.2 Self-review and external feedback 5
2.3 Local inference and structured reporting 5

3 Problem formulation **6**
3.1 Approved contract and report 6
3.2 Supported arithmetic 6

4 System design **7**
4.1 Independent computation and field checks 7
4.2 Repair paths and demonstration 7

5 Experimental design **8**
5.1 Datasets and split 8
5.2 Controlled mutations 8
5.3 Actual-model comparison 9
5.4 Metrics and uncertainty 9

6 Results and error analysis **10**
6.1 Checking layers and controlled coverage 10
6.2 Model outcomes 11
6.3 What deterministic execution establishes 13

7 Threats to validity and limitations **13**
7.1 Internal validity 13
7.2 Construct and external validity 14
7.3 Practical assurance 14

8 Reproducibility and software quality **14**

9 Conclusion **15**

1 Introduction

Language models provide an accessible interface for data analysis, but a fluent explanation does not establish that its calculations are correct. A report may use the wrong denominator, count missing observations, attach a wrong unit, or plot values different from those in its numeric result. These failures matter even in basic descriptive analysis. The central engineering question is how much of a candidate analysis can be checked with executable evidence and how precisely the resulting assurance should be described.

This project studies a bounded setting. A human approves the operation, columns, unit and missing-value rule before verification. A model proposes a structured report. TrustVerify recomputes the approved operation and returns field-specific findings. The model can then receive those findings in one repair call. A separate deterministic path replaces the checked fields directly. Neither path certifies free-form prose or discovers the correct research question.

Three research questions organize the study. RQ1 asks which error classes different checking layers detect. RQ2 asks whether executable feedback improves structured correctness compared with an equal-call self-review condition. RQ3 asks what role remains for the model when the analysis contract is already explicit. The third question is essential: direct computation is already sufficient for the six implemented operations. Any successful tool condition must not be misrepresented as evidence that an LLM learned to reason more accurately.

The contributions are a dependency-free Python prototype, a reproducible synthetic benchmark with an independent oracle, a controlled ablation, actual local-model response logs, and a demonstration of both successful checks and failure boundaries. This is a course-scale engineering and evaluation study. External feedback and arithmetic verification are established ideas; the work does not claim a new general verification theory or a new language model.

2 Related work

2.1 Evaluating data-science systems

DSBench evaluates data-science agents on realistic tasks involving substantial files and multi-stage analysis[1]. It motivates examining observed outcomes rather than treating generated analysis as completed work. Our synthetic benchmark has a different, narrower purpose: exact control over the operation, missingness, source values and injected fault. Results from this project are not directly comparable with DSBench scores and do not measure end-to-end data-science competence.

2.2 Self-review and external feedback

Self-Refine studies iterative improvement using feedback from the same model without additional model training[2]. Its task-dependent results motivate including a self-review baseline. Huang et al. investigate limitations of intrinsic self-correction in reasoning[3]. Their findings motivate an independent computational signal, but do not establish that self-review always fails or that every form of external feedback succeeds. This project evaluates those alternatives under one explicit local protocol and preserves negative results.

2.3 Local inference and structured reporting

Qwen2.5 provides models of different capacities[4]. We use quantized 0.5B and 1.5B variants through Ollama's chat interface[5]. The experiment requests JSON output but does not impose a strict JSON schema. Consequently, syntactically valid JSON can still violate the report contract. These small local models make the study accessible, but their results do not estimate the performance of larger hosted systems.

3 Problem formulation

3.1 Approved contract and report

Let D be a CSV dataset, C the human-approved contract and R a candidate JSON report. The contract specifies one of six operations, the required columns and the output unit. The report contains a metric mapping, a unit string, an integer valid-row count, chart labels and numeric values, and a narrative string. The verifier computes $E = f_C(D)$ and compares the structured fields of R with E.

For a scalar answer, the metric key is `value`. A grouped answer uses the group labels as keys. Chart values must match their named metrics, although the chart label order can vary. A field passes numeric comparison when

$$|r - e| \leq \max\left(10^{-6}, 10^{-6}\max(|r|, |e|)\right). \tag{3-1}$$

Booleans, numeric strings and non-finite values do not count as numeric answers. This tolerance accommodates output rounding. It also defines a detectable-error boundary: smaller differences can pass.

3.2 Supported arithmetic

The operations are sum, arithmetic mean, weighted mean, ratio of sums expressed as a percentage, group sum and first-to-last percentage change. Weighted mean is

$$\bar{x}_w = \frac{\sum_{i\in I} w_i x_i}{\sum_{i\in I} w_i}, \tag{3-2}$$

where I includes rows with both values present. The percentage ratio is

$$p = 100\frac{\sum_{i\in I} x_i}{\sum_{i\in I} n_i}. \tag{3-3}$$

It is deliberately defined as a ratio of sums, which differs from averaging row-wise ratios. Percentage change uses the first and last nonmissing observations in CSV order. The engine does not infer chronological order from column names.

Missing cells are excluded according to the columns required by the operation. All duplicate records count as separate observations. The approved unit is used literally;

dimensional analysis is outside scope. Empty data, missing columns, unsupported operations, zero aggregate denominators and invalid weights produce an abstention. The public API distinguishes `pass`, `fail` and `abstain` so an unsupported task does not silently receive a successful verdict.

4 System design

4.1 Independent computation and field checks

The production engine uses Python's Decimal arithmetic with a 40-digit calculation context and emits finite JSON numbers. It never reads evaluation gold answers. The independent evaluation oracle uses SQLite aggregation and separate query logic. Agreement between these implementations checks common arithmetic mistakes, although it cannot eliminate all shared specification errors.

The numeric profile checks metric labels and values. The schema-plus-numeric profile also requires the expected top-level fields and selected field types. Full verification adds unit equality, complete-case row count and chart consistency. These profiles provide an ablation over coverage rather than three independently trained systems.

Each finding contains a code, field path, explanation and expected/observed values when available. For example, an incorrect percentage can yield a finding for `metrics.value` with the recomputed percentage. Such expected values are derived from the available data and approved contract. They are computational feedback, not hidden benchmark labels, but they do give the model more information than self-review receives.

4.2 Repair paths and demonstration

The model repair path receives the initial response and checker findings in one additional call. The deterministic path recomputes the report without another model call. It replaces the narrative with an explicit statement that no causal conclusion has been established. We label this path deterministic recomputation throughout the results.

The local demonstration serves a task selector, an editable candidate report, the approved contract and original rows. Users can inject a named fault, run an ablated checker

and recompute the structured report. All examples are visibly identified as controlled cases. The application binds to loopback, accepts bounded JSON input and executes no submitted Python or shell code. It is a local prototype, not a general code-execution sandbox.

5 Experimental design

5.1 Datasets and split

The generator creates ten datasets for each operation, using independently determined seeds. Every dataset has 12 rows and three columns: numeric observation x, weight or denominator n, and group g. Two observations of x and one of n are missing in each dataset. Variants 00–01 form 12 development tasks, and 02–09 form 48 test tasks. The model pilot uses variants 02–05, providing four test tasks per family and 24 per model. Each task manifest includes a CSV SHA-256 digest.

This is a dataset-level split within six fixed task templates. It does not test transfer to unseen operation families. All observations are synthetic and carry no substantive claim about real people, institutions or economic outcomes.

5.2 Controlled mutations

For each of the 48 test tasks we retain a correct report and construct seven single-fault reports. Six mutations target supported fields: a metric increased by 3.25, a wrong unit, a valid-row count increased by one, a chart value increased by five, a missing chart and a metric replaced by the string NaN. The seventh inserts an unsupported causal narrative while preserving every structured value.

The resulting experiment contains 48 clean and 336 faulty reports, evaluated under each profile. Mutation labels never enter the checker. Faults sharing one dataset are dependent, and the types are intentionally chosen. Therefore, their detection proportions describe coverage of the injected fault set, not the natural frequency or population prevalence of LLM errors. We do not attach naive binomial confidence intervals to these mutation counts.

5.3 Actual-model comparison

The initial pilot uses Qwen2.5 0.5B. After observing frequent schema failures, we disclose a capacity-check amendment and run Qwen2.5 1.5B on the same tasks with unchanged prompts. This is a post-pilot capacity check rather than a preregistered independent replication. Both models use temperature zero, seed 31415, context limit 4096 and a maximum of 384 generated tokens per call. Exact model digests, prompts and provider settings are saved.

For every task an initial call generates a report. Two separate branches then start from that same initial report. One receives a self-review instruction with the data and question; the other receives executable checker findings. Each branch gets exactly one additional call with the same output-token limit. Thus each model contributes 72 calls: 24 initial, 24 self-review and 24 feedback repair. The branches have equal call count but differ in prompt length and available computational information.

Invalid JSON or an invalid report is a measured content failure. Service/network failures stop the run rather than being scored as model mistakes. Raw completed calls are retained, and resuming a run checks request hashes. There is no content retry or selection of a preferred response. Generated code is never executed.

5.4 Metrics and uncertainty

Primary correctness requires all metric labels/values, units, valid-row count and chart data to match the independent oracle within tolerance. Narrative truth is excluded from this metric. We also record JSON-object validity, field-level findings, wall-clock latency and provider token counts. Model tables report cumulative initial-plus-second-call latency for review and feedback. Deterministic computation time is reported separately.

We give Wilson 95% intervals for each model condition and a task-level paired bootstrap interval for feedback-minus-self-review accuracy, using 10,000 resamples and fixed seed 20261110. These intervals describe a small, clustered-by-template synthetic task sample. They do not establish population-wide model performance or cross-model superiority.

6 Results and error analysis

6.1 Checking layers and controlled coverage

Numeric-only verification detects 96 of 336 injected faults (28.6%). Requiring the report schema increases detection to 144 (42.9%). Full structured checking detects 288 (85.7%). All profiles produce zero false alarms on the 48 clean reports. Full verification detects all 288 supported structured mutations, while missing all 48 causal-narrative mutations. Figure 6-1 shows the aggregate ablation and Figure 6-2 separates the fault categories.

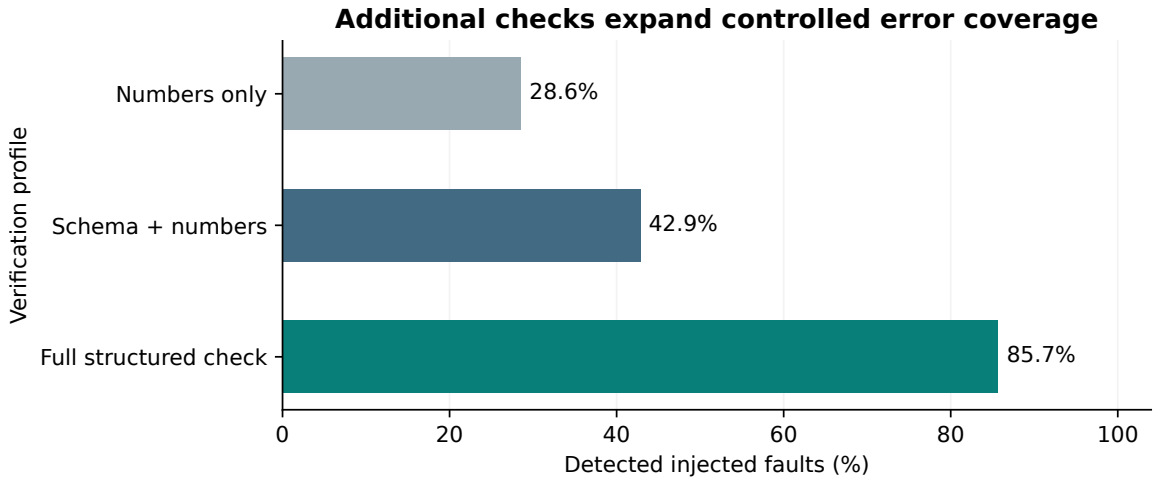


Figure 6-1 Detection of controlled injected faults. The denominator includes the intentionally unsupported narrative mutation. These are checker-coverage results, not natural model error rates.

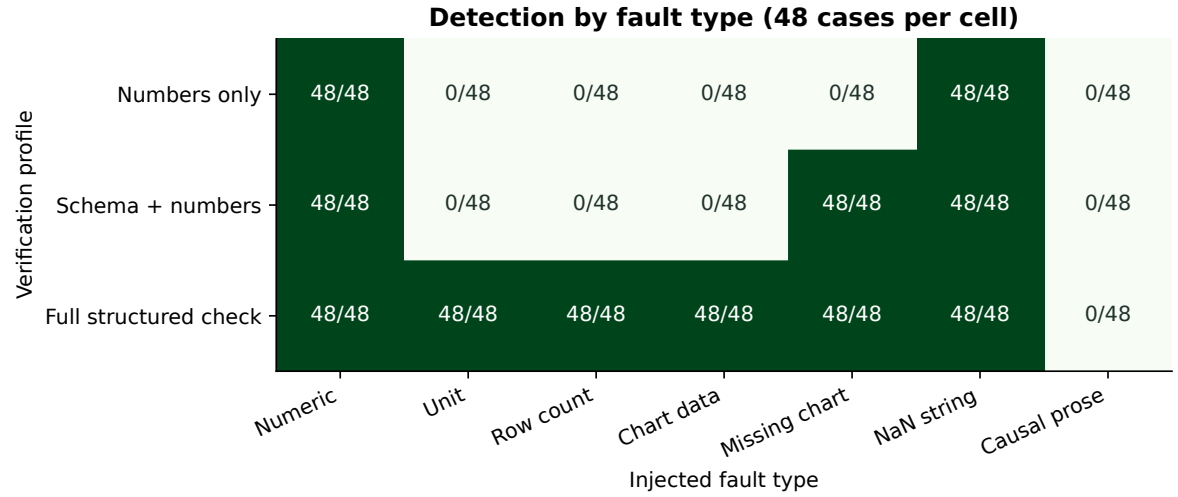


Figure 6-2 Detection counts by field and profile. Each cell contains 48 controlled cases.

This result is consistent with the design: numeric checking cannot discover a wrong unit when the metric itself is unchanged. Schema checking can detect the missing chart but not a present chart with wrong values. Full checking adds independent evidence for those fields. However, because the mutations directly exercise designed checks, perfect supported-fault detection is a coverage observation, not proof against arbitrary failures.

6.2 Model outcomes

Table 6-1 Measured actual-model pilot. Each generation condition uses 24 paired tasks. Intervals are descriptive Wilson 95% intervals for structured correctness.

Model	Condition	Correct	95% interval (%)	Time (s)
0.5B	Initial	0/24	0.0–13.8	5.81
0.5B	Self-review	0/24	0.0–13.8	11.14
0.5B	Feedback repair	0/24	0.0–13.8	11.48
1.5B	Initial	0/24	0.0–13.8	7.08
1.5B	Self-review	0/24	0.0–13.8	14.14
1.5B	Feedback repair	0/24	0.0–13.8	15.29

For the 0.5B model, executable feedback minus self-review is 0.0 percentage points (paired bootstrap 95% interval 0.0 to 0.0). There are 0 improved and 0 regressed paired tasks. For the 1.5B model, executable feedback minus self-review is 0.0 percentage points (paired bootstrap 95% interval 0.0 to 0.0). There are 0 improved and 0 regressed paired tasks. These descriptive intervals do not establish general equivalence or superiority. In a floor-effect condition, a zero-width empirical bootstrap interval reflects no observed successes in either branch, not certainty about unseen tasks.

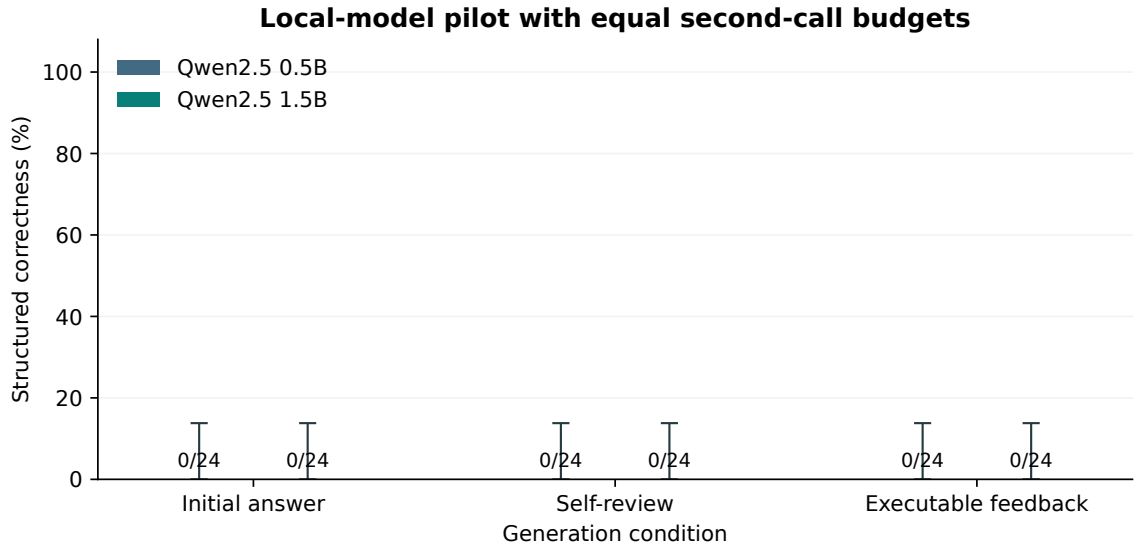


Figure 6-3 Observed structured correctness for actual local-model responses. Whiskers show Wilson 95% intervals. Deterministic recomputation scores 24/24 separately for each task set.

The 0.5B model produces JSON objects on 19 of 24 tasks in every generation condition, but none satisfies the full structured correctness criterion. On task sum-02, it reports a value of 10, embeds unit and row-count fields inside the metric mapping, and uses chart labels that do not correspond to the expected single metric. Self-review and feedback repair preserve these problems. This example illustrates that a parseable object is weaker than a valid, correct report. The unchanged prompt and retained responses make this floor effect visible instead of silently removing it from the evaluation.

An exploratory decomposition for the 1.5B model finds correct numeric metric mappings on 0/24 initial, 0/24 self-review and 19/24 feedback-repair responses. This sec-

ondary diagnostic excludes units, row counts and chart fields and must not replace the primary correctness score. It helps distinguish arithmetic repair from complete-report repair. The field-level finding counts are preserved in `secondary_diagnostics.json`.

6.3 What deterministic execution establishes

Deterministic recomputation matches the independent oracle on all 60 generated tasks, including all 48 held-out checker tasks and the 24-task pilot subset. This is an expected consequence of correctly implementing an explicit operation. It provides a useful deployment path for supported contracts, but does not constitute language-model reasoning improvement. It also makes the project's arithmetic boundary clear: when the operation is known, conventional software is the appropriate source of the result.

The model may still be useful for proposing a readable interface or a narrative, but those roles require additional validation. Our reported correctness metric cannot evaluate them. It would be misleading to translate 24/24 deterministic accuracy into a claim that the entire assistant is fully reliable.

7 Threats to validity and limitations

7.1 Internal validity

Using Decimal and SQLite reduces identical-implementation bias, but both follow the same written contract. A wrong contract can make both implementations agree on the wrong task. Likewise, final scoring shares a numeric-tolerance policy with checking. Hand-calculated unit tests provide an additional independent check for representative operations.

The feedback branch receives expected values computed from the data, while self-review does not. Consequently, the experiment compares complete systems with different evidence access. Equal generation counts do not make available information, token counts or total computation identical. Cold-start loading, caching and hardware scheduling affect measured latency. Wall-clock results are local observations, not general speed benchmarks.

7.2 Construct and external validity

Structured correctness excludes narrative accuracy, causal validity, input-data quality, visual chart design and choice of statistical method. In the causal-claim mutation, all structured fields can pass while the report makes an unjustified assertion. In a second scope challenge, changing an approved sum contract to a mean contract and computing that mean correctly also passes. The verifier has no evidence of the user's original intent.

The synthetic tasks are small, have fixed templates and cover only six descriptive operations. Two small quantized models and one generation seed offer limited generalization. The larger model was selected after inspecting the smaller pilot, which we disclose. Bootstrap uncertainty over 24 tasks does not account for all model, dataset, hardware or prompt variation. Wider evaluations should add independently sourced real datasets, unseen task families and stronger models while retaining these original results.

7.3 Practical assurance

A pass should read "checked structured fields agree with this approved contract", never "the report is true". Unsupported tasks should route to human review. A practical extension would ask a user to approve the proposed contract and inspect data-quality decisions before generation. Free-text claims could receive a separate, calibrated evidence review. These extensions are future work and have not been implemented or evaluated here.

8 Reproducibility and software quality

The core package uses Python 3.10 or later with no third-party runtime dependencies. The repository includes 60 CSVs, their task manifests and hashes, independent oracle values, controlled fault records, model-call logs, summary statistics, figure source and the report source. Tests cover hand calculations, missingness, undefined contracts, malformed reports, chart consistency, repair behavior, known blind spots and oracle agreement.

The main reproduction commands are listed below. They run from the repository root.

```
python3 -m unittest discover -s tests -v
python3 -m trustverify.cli generate
python3 -m trustverify.cli benchmark
python3 scripts/check_results.py
python3 -m trustverify.server
```

The first command runs the tests, the second regenerates data, the third reproduces controlled results, the fourth checks saved model evidence without new calls, and the last starts the demonstration. The README gives full model and manuscript instructions.

The report adapts the supplied SCU format, preserving A4 geometry, bilingual abstracts, section numbering and GB/T 7714 references. The supplied licence file is AGPL-3.0 despite a GPL reference in its README; the original file and attribution are retained. Lecturer slides are identified in a source manifest and are not redistributed.

9 Conclusion

TrustVerify provides executable evidence for a deliberately limited set of analysis claims. The ablation shows how schema, unit, row-count and chart checks expand coverage beyond numeric comparison. The causal-narrative and wrong-contract examples demonstrate why that coverage cannot establish whole-report truth. In the 1.5B pilot, an exploratory decomposition finds that executable feedback produces 19/24 correct numeric mappings, compared with 0/24 for self-review, while full-report correctness remains 0/24 in both branches. The actual-model comparison therefore exposes the gap between arithmetic repair and complete-report compliance. The deterministic baseline shows that conventional computation already solves approved descriptive contracts. The practical outcome is a reproducible checking tool with inspectable failure boundaries.

AI assistance and author responsibility. Codex assisted with implementation, experiment execution, drafting and layout. Reported model outputs are preserved local Ollama calls; controlled errors are synthetic. The author must reproduce and understand the work before submission. No instructor approval or grade is claimed.

参考文献

[1] JING L, HUANG Z, WANG X, et al. DSBench: How far are data science agents from becoming data science experts?[A/OL]. 2025. https://arxiv.org/abs/2409.07703v3.

[2] MADAAN A, TANDON N, GUPTA P, et al. Self-refine: Iterative refinement with self-feedback [A/OL]. 2023. https://arxiv.org/abs/2303.17651.

[3] HUANG J, CHEN X, MISHRA S, et al. Large language models cannot self-correct reasoning yet[A/OL]. 2024. https://arxiv.org/abs/2310.01798v2.

[4] Qwen Team. Qwen2.5: A party of foundation models![EB/OL]. 2024. https://qwenlm.github.io/blog/qwen2.5/.

[5] Ollama contributors. Ollama documentation: Generate a chat message[EB/OL]. 2026. https://docs.ollama.com/api/chat.